



SISTEMA DE VAREJO SIMPLES

Persistência de Dados

Marco Antonio do Bomfim Mira ^{*1}, Caiane ^{†1}, Angela Peixoto
Santana ^{‡1*}

¹ Banco de Dados | Turma: ESW-NOT-PIT-3S-T1 - 53010

Escola de Tecnologias

Universidade Católica do Salvador (UCSAL)

Av. Prof. Pinto de Aguiar, 2589 Pituaçu, CEP: 41740-090

Salvador/BA, Brasil

¹*marco.mira@ucsal.edu.br - Matrícula: 200037033*

¹*caiane.santos@ucsal.edu.br - Matrícula: 00000000*

¹*angela.santana@pro.ucsal.edu.br*

Junho 2026

Resumo

O presente trabalho apresenta o desenvolvimento de um Sistema de Varejo de Produtos, elaborado como atividade prática para a disciplina de Banco de Dados I. O projeto tem como objetivo demonstrar a persistência de dados utilizando a linguagem Java integrada ao SGBD PostgreSQL através da API JDBC.

^{*}marco.mira@ucsal.edu.br

[†]caiane.santos@ucsal.edu.br

[‡]angela.santana@pro.ucsal.br

O sistema contempla funcionalidades essenciais para gerenciamento de varejo, como cadastro de clientes, fornecedores, categorias e produtos, além do controle de compras, vendas e movimentação automática de estoque através da emissão de notas de compra e venda.

A aplicação foi estruturada utilizando arquitetura em camadas simples, separando responsabilidades entre menus, serviços, repositórios e modelos, permitindo maior organização e manutenção do código. O projeto também utiliza recursos avançados do PostgreSQL, como views, constraints e relacionamentos entre tabelas, garantindo integridade e consistência dos dados.

O desenvolvimento do sistema reforça conceitos fundamentais de persistência de dados, modelagem relacional, operações CRUD, controle de estoque e integração entre banco de dados e programação orientada a objetos.

Palavras-chaves: 1. Sistema de Varejo 2. PostgreSQL 3. Java-JDBC 4. Persistência de Dados.

1 Introdução

Este projeto consiste no desenvolvimento de um Sistema de Varejo de Produtos, proposto como atividade avaliativa para a disciplina de Banco de Dados. O principal objetivo é aplicar conceitos de modelagem de banco de dados, persistência de dados e integração entre aplicações Java e o SGBD PostgreSQL utilizando JDBC.

O sistema foi desenvolvido para simular operações básicas de um ambiente de varejo, permitindo o cadastro de clientes, fornecedores, categorias e produtos, além do gerenciamento de compras e vendas com cashback para posteriores compras, garantindo a regra de negócio estabelecida, que após uma venda será gerado um valor de cashback a ser armazenado. O fluxo operacional realiza automaticamente o controle de estoque através das movimentações de entrada e saída de produtos vinculadas às notas de compra e venda.

A arquitetura da aplicação foi organizada em camadas simples, separando responsabilidades entre menus, serviços, modelos e repositórios, permitindo melhor manutenção e reutilização do código. Também foram aplicadas boas práticas de banco de dados utilizando chaves primárias, chaves estrangeiras, constraints e views para consultas.

O sistema possui entidades principais como Cliente, Fornecedor, Produto, Categoria, Nota de Compra, Nota de Venda, Item de Compra e Item de Venda, representando um fluxo real de operações comerciais em um ambiente varejista.

Esse projeto possui finalidade exclusivamente educacional, visando reforçar de forma prática os conceitos de banco de dados relacionais, persistência de dados e desenvolvimento de aplicações Java conectadas ao PostgreSQL.

2 Banco de Dados

2.1 Modelos de Banco de Dados

2.1.1 Modelo Conceitual do Sistema de Varejo

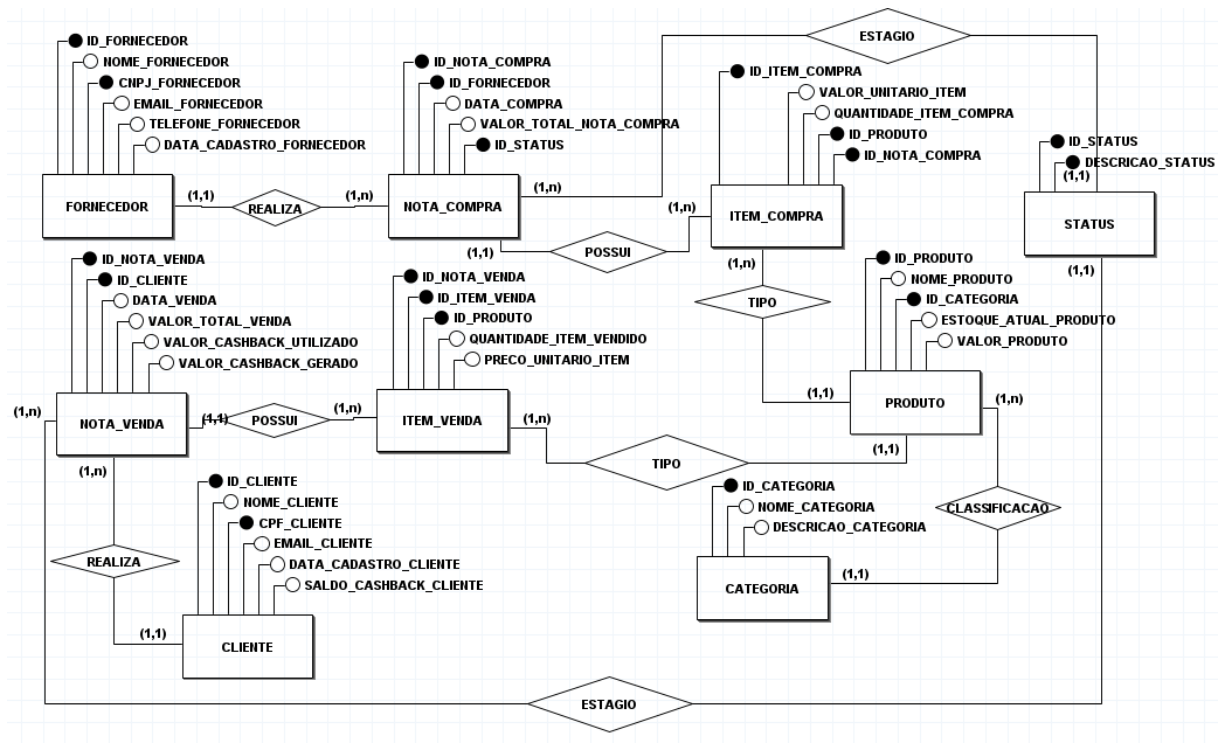


Figura 1 – Diagrama Modelo Conceitual

O modelo conceitual do Sistema de Varejo é composto pelas entidades principais: **Cliente**, **Fornecedor**, **Categoria**, **Produto**, **Nota de Compra**, **Item de Compra**, **Nota de Venda** e **Item de Venda**.

- **Cliente** armazena os dados dos clientes responsáveis pelas compras realizadas no sistema.
- **Fornecedor** representa as empresas fornecedoras responsáveis pelo abastecimento do estoque.
- **Categoria** organiza os produtos em grupos para melhor controle e gerenciamento.
- **Produto** contém informações dos produtos comercializados, como nome, preço e quantidade em estoque.
- **Nota de Compra** registra as operações de entrada de produtos no estoque através dos fornecedores.

- **Item de Compra** representa os produtos vinculados às notas de compra, armazenando quantidade e preço unitário.
- **Nota de Venda** registra as vendas realizadas para clientes.
- **Item de Venda** representa os produtos vendidos em cada nota de venda.

Os relacionamentos garantem o controle correto do estoque através das movimentações de entrada e saída de produtos, mantendo a integridade dos dados e simulando um fluxo real de operações varejistas.

2.1.2 Modelo Lógico do Sistema de Varejo

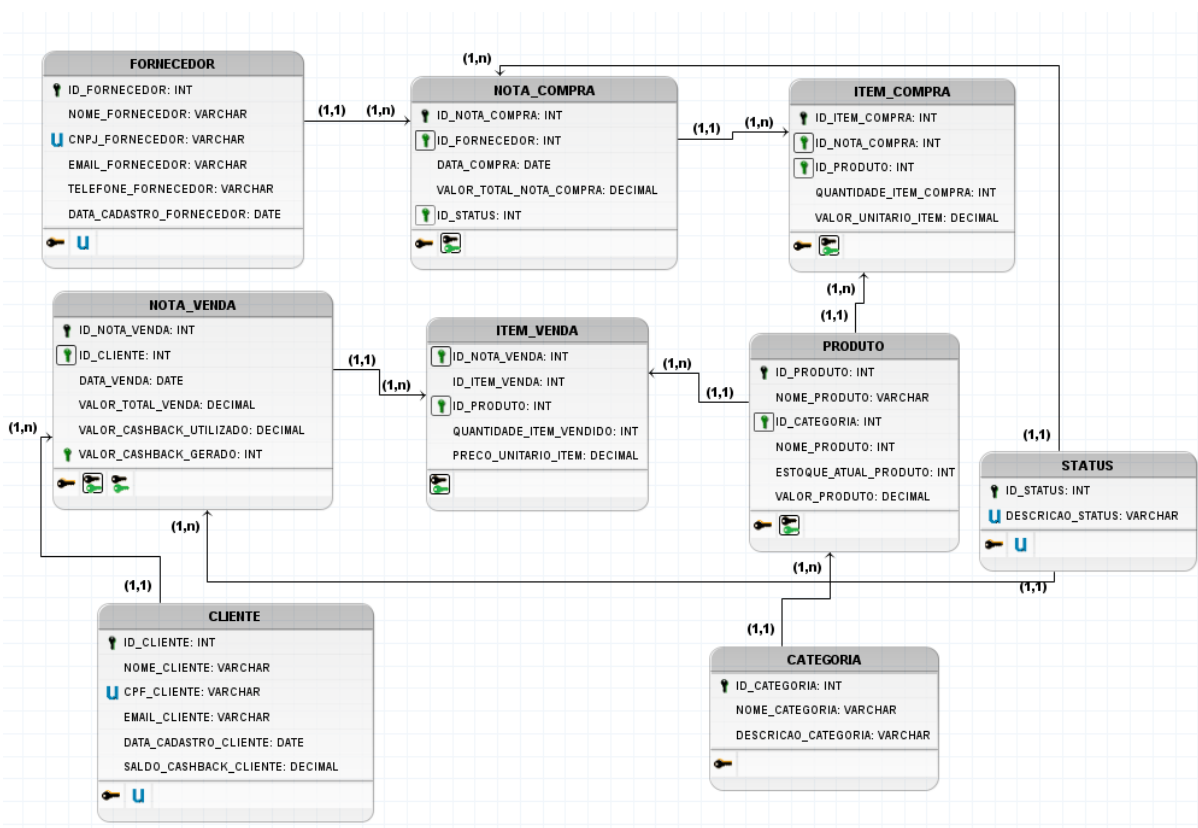


Figura 2 – Diagrama Modelo Lógico

O modelo lógico do Sistema de Varejo representa a estrutura das tabelas no banco de dados relacional, contendo suas chaves primárias, chaves estrangeiras, atributos e relacionamentos necessários para o controle de compras, vendas, clientes, fornecedores, produtos, categorias e status.

- A tabela **Fornecedor** armazena os dados dos fornecedores responsáveis pelo abastecimento dos produtos. Possui como chave primária o campo **id_fornecedor** e contém informações como nome, CNPJ, e-mail, telefone e data de cadastro.

- A tabela **Cliente** armazena os dados dos clientes que realizam compras no sistema. Possui como chave primária o campo **id_cliente**, além de dados como nome, CPF, e-mail, data de cadastro e saldo de cashback.
- A tabela **Categoria** organiza os produtos em grupos. Possui como chave primária o campo **cod_categoria**, além do nome e descrição da categoria.
- A tabela **Produto** representa os produtos comercializados no sistema. Possui como chave primária o campo **id_produto** e uma chave estrangeira **id_categoria**, relacionando cada produto a uma categoria. Também armazena nome, estoque atual e valor do produto.
- A tabela **Status** armazena os possíveis estados das notas, como processamento, concluído ou cancelado. Possui como chave primária o campo **id_status** e a descrição do status.
- A tabela **Nota_Compra** registra as compras realizadas junto aos fornecedores. Possui como chave primária o campo **id_nota_compra** e chaves estrangeiras para **Fornecedor** e **Status**. Também registra a data da compra e o valor total da nota.
- A tabela **Item_Compra** detalha os produtos comprados em cada nota de compra. Possui como chave primária o campo **id_item_compra** e chaves estrangeiras para **Nota_Compra** e **Produto**. Armazena a quantidade comprada e o valor unitário do item.
- A tabela **Nota_Venda** registra as vendas realizadas para os clientes. Possui como chave primária o campo **id_nota_venda** e chave estrangeira para **Cliente**. Armazena a data da venda, valor total, cashback utilizado e cashback gerado.
- A tabela **Item_Venda** detalha os produtos vendidos em cada nota de venda. Possui como chave primária o campo **id_item_venda** e chaves estrangeiras para **Nota_Venda** e **Produto**. Armazena a quantidade vendida e o preço unitário do item.

Os relacionamentos do modelo garantem a integridade dos dados e permitem controlar o fluxo completo do varejo. Um fornecedor pode estar relacionado a várias notas de compra, uma nota de compra pode possuir vários itens de compra, uma categoria pode possuir vários produtos, um cliente pode realizar várias vendas e uma nota de venda pode conter vários itens vendidos.

Esse modelo lógico serve como base para a implementação física do banco de dados no PostgreSQL, permitindo o controle de estoque, registro de compras, registro de vendas, cálculo de cashback e organização das informações comerciais do sistema.

3 Implementação Física do Banco de Dados

Após a construção do modelo conceitual e lógico, foi realizada a implementação física do banco de dados utilizando o Sistema Gerenciador de Banco de Dados PostgreSQL.

Nesta etapa foram criadas as tabelas, relacionamentos, constraints, views, procedures e functions responsáveis pelo funcionamento do Sistema de Varejo de Produtos.

A modelagem física foi desenvolvida com o objetivo de garantir integridade, organização, consistência e automatização das operações realizadas pelo sistema, principalmente no controle de estoque, compras, vendas e gerenciamento de clientes e fornecedores.

O banco de dados foi estruturado seguindo o modelo relacional, utilizando chaves primárias, chaves estrangeiras e regras de integridade referencial para garantir a correta comunicação entre as tabelas.

3.1 Script DDL (Data Definition Language)

3.1.1 Script para criação do banco de dados

```
1  CREATE DATABASE "EmpresaVarejo"
2  WITH
3  OWNER = postgres
4  ENCODING = 'UTF8'
5  LC_COLLATE = 'Portuguese_Brazil.1252'
6  LC_CTYPE = 'Portuguese_Brazil.1252'
7  LOCALE_PROVIDER = 'libc'
8  TABLESPACE = pg_default
9  CONNECTION LIMIT = -1
10 IS_TEMPLATE = False;
11
```

Figura 3 – Script CREATE DATABASE

A Figura 3 ilustra a estrutura utilizada para desenvolver o script de criação do banco no PostgreSQL.

3.2 Criação das Tabelas e Constraints

As tabelas do sistema foram criadas utilizando comandos DDL (*Data Definition Language*), responsáveis pela definição da estrutura física do banco de dados.

```

59  --TABLE CATEGORIA
60  CREATE TABLE IF NOT EXISTS public.categoria
61  (
62      cod_categoria integer NOT NULL GENERATED ALWAYS AS IDENTITY (
63          INCREMENT 1 START 1 MINVALUE 1 MAXVALUE 2147483647 CACHE 1 ),
64      nome_categoria character varying(100) COLLATE pg_catalog."default"
65          NOT NULL,
66      descricao character varying(300) COLLATE pg_catalog."default" NOT
67          NULL,
68      CONSTRAINT categoria_pkey PRIMARY KEY (cod_categoria),
69      CONSTRAINT categoria_nome_categoria_key UNIQUE (nome_categoria)
70  );
71
72  --TABLE CLIENTE
73  CREATE TABLE IF NOT EXISTS public.cliente
74  (
75      id_cliente integer NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT
76          1 START 1 MINVALUE 1 MAXVALUE 2147483647 CACHE 1 ),
77      nome_cliente character varying(100) COLLATE pg_catalog."default" NOT
78          NULL,
79      cpf_cliente character varying(11) COLLATE pg_catalog."default" NOT
80          NULL,
81      email_cliente character varying(100) COLLATE pg_catalog."default"
82          NOT NULL,
83      data_cadastro date NOT NULL,
84      saldo_cashback numeric DEFAULT 0,
85      CONSTRAINT cliente_pkey PRIMARY KEY (id_cliente),
86      CONSTRAINT cliente_cpf_cliente_key UNIQUE (cpf_cliente)
87  );

```

Figura 4 – Entidades Categoria e Cliente

```

81  --TABLE FORNECEDOR
82  CREATE TABLE IF NOT EXISTS public.fornecedor
83  (
84      id_fornecedor integer NOT NULL GENERATED ALWAYS AS IDENTITY (
85          INCREMENT 1 START 1 MINVALUE 1 MAXVALUE 2147483647 CACHE 1 ),
86      nome_fornecedor character varying(100) COLLATE pg_catalog."default"
87          NOT NULL,
88      cnpj_fornecedor character varying(14) COLLATE pg_catalog."default"
89          NOT NULL,
90      email_fornecedor character varying(100) COLLATE pg_catalog."default"
91          NOT NULL,
92      telefone_fornecedor character varying(20) COLLATE pg_catalog.
93          "default",
94      data_cadastro_fornecedor date NOT NULL,
95      CONSTRAINT fornecedor_pkey PRIMARY KEY (id_fornecedor),
96      CONSTRAINT fornecedor_cnpj_fornecedor_key UNIQUE (cnpj_fornecedor)
97  );
98
99  --TABLE ITEM_COMPRA
100  CREATE TABLE IF NOT EXISTS public.item_compra
101  (
102      id_item_compra integer NOT NULL GENERATED ALWAYS AS IDENTITY (
103          INCREMENT 1 START 1 MINVALUE 1 MAXVALUE 2147483647 CACHE 1 ),
104      quantidade_produto_comprado integer NOT NULL,
105      valor_unitario_item numeric NOT NULL,
106      id_nota_compra integer NOT NULL,
107      id_produto integer NOT NULL,
108      CONSTRAINT item_compra_id_nota_compra_fkey FOREIGN KEY
109          (id_nota_compra)
110          REFERENCES public.nota_compra (id_nota_compra) MATCH SIMPLE
111          ON UPDATE NO ACTION
112          ON DELETE NO ACTION,
113      CONSTRAINT item_compra_id_produto_fkey FOREIGN KEY (id_produto)
114          REFERENCES public.produto (id_produto) MATCH SIMPLE
115          ON UPDATE NO ACTION
116          ON DELETE NO ACTION
117  );

```

Ativar o Windows

Figura 5 – Entidades Fornecedor e Item Compra


```

112  --TABLE ITEM_VENDA
113  CREATE TABLE IF NOT EXISTS public.item_venda
114  (
115      id_item_venda integer NOT NULL GENERATED ALWAYS AS IDENTITY (
116          INCREMENT 1 START 1 MINVALUE 1 MAXVALUE 2147483647 CACHE 1 ),
117      quantidade_produto_vendido integer NOT NULL,
118      preco_unitario_venda numeric NOT NULL,
119      id_produto integer NOT NULL,
120      id_nota_venda integer NOT NULL,
121      CONSTRAINT item_venda_pkey PRIMARY KEY (id_item_venda),
122      CONSTRAINT item_venda_id_nota_venda_fkey FOREIGN KEY (id_nota_venda)
123          REFERENCES public.nota_venda (id_nota_venda) MATCH SIMPLE
124          ON UPDATE NO ACTION
125          ON DELETE NO ACTION,
126      CONSTRAINT item_venda_id_produto_fkey FOREIGN KEY (id_produto)
127          REFERENCES public.produto (id_produto) MATCH SIMPLE
128          ON UPDATE NO ACTION
129          ON DELETE NO ACTION
130  );
131
132  --TABLE NOTA_COMPRA
133  CREATE TABLE IF NOT EXISTS public.nota_compra
134  (
135      id_nota_compra integer NOT NULL GENERATED ALWAYS AS IDENTITY (
136          INCREMENT 1 START 1 MINVALUE 1 MAXVALUE 2147483647 CACHE 1 ),
137      id_fornecedor integer NOT NULL,
138      data_compra date NOT NULL,
139      valor_compra numeric NOT NULL,
140      id_status integer DEFAULT 1,
141      CONSTRAINT nota_compra_pkey PRIMARY KEY (id_nota_compra),
142      CONSTRAINT id_status FOREIGN KEY (id_status)
143          REFERENCES public.status (id_status) MATCH SIMPLE
144          ON UPDATE CASCADE
145          ON DELETE NO ACTION,
146      CONSTRAINT nota_compra_id_fornecedor_fkey FOREIGN KEY (id_fornecedor)
147          REFERENCES public.fornecedor (id_fornecedor) MATCH SIMPLE
148          ON UPDATE NO ACTION
149          ON DELETE NO ACTION
150  );

```

Ativar o Windows

Figura 6 – Entidades Item Venda e Nota Compra

```

150  --TABLE NOTA_VENDA
151  CREATE TABLE IF NOT EXISTS public.nota_venda
152  (
153      id_nota_venda integer NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT
154      1 START 1 MINVALUE 1 MAXVALUE 2147483647 CACHE 1 ),
155      id_cliente integer NOT NULL,
156      data_venda date NOT NULL,
157      valor_total_venda numeric NOT NULL,
158      valor_cashback_gerado numeric NOT NULL,
159      valor_cashback_utilizado numeric DEFAULT 0,
160      id_status integer DEFAULT 1,
161      CONSTRAINT nota_venda_pkey PRIMARY KEY (id_nota_venda),
162      CONSTRAINT id_status FOREIGN KEY (id_status)
163          REFERENCES public.status (id_status) MATCH SIMPLE
164          ON UPDATE CASCADE
165          ON DELETE NO ACTION,
166      CONSTRAINT nota_venda_id_cliente_fkey FOREIGN KEY (id_cliente)
167          REFERENCES public.cliente (id_cliente) MATCH SIMPLE
168          ON UPDATE NO ACTION
169          ON DELETE NO ACTION
170  );
171
172  --TABLE PRODUTO
173  CREATE TABLE IF NOT EXISTS public.produto
174  (
175      id_produto integer NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT 1
176      START 1 MINVALUE 1 MAXVALUE 2147483647 CACHE 1 ),
177      id_categoria integer NOT NULL,
178      nome_produto character varying(50) COLLATE pg_catalog."default" NOT
179      NULL,
180      preco_unitario numeric NOT NULL,
181      estoque_atual integer DEFAULT 0,
182      CONSTRAINT produto_pkey PRIMARY KEY (id_produto)
183  );

```

Figura 7 – Entidades Nota Venda e Produto

```

182  --TABLE STATUS
183  CREATE TABLE IF NOT EXISTS public.status
184  (
185      id_status integer NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT 1
186      START 1 MINVALUE 1 MAXVALUE 2147483647 CACHE 1 ),
187      descricao character varying(14) COLLATE pg_catalog."default" NOT NULL,
188      CONSTRAINT pk_status PRIMARY KEY (id_status)
189  );
190

```

Figura 8 – Entidade Status

As principais tabelas implementadas no sistema foram:

- Cliente
- Fornecedor
- Categoria
- Produto
- Nota Compra
- Item Compra
- Nota Venda
- Item Venda
- Status

Durante a implementação também foram utilizadas **constraints**, responsáveis por garantir a integridade e consistência das informações armazenadas no banco de dados.

As principais constraints utilizadas foram:

- **PRIMARY KEY**: utilizada para identificação única dos registros. ““
- **FOREIGN KEY**: utilizada para criação dos relacionamentos entre tabelas.
- **UNIQUE**: utilizada para impedir duplicidade em campos como CPF, CNPJ e nome de categoria.
- **DEFAULT**: utilizada para definir valores automáticos, como saldo inicial de cash-back e status padrão das notas. ““

As tabelas utilizam identificadores automáticos do tipo *IDENTITY*, permitindo geração automática dos IDs dos registros.

3.3 Uso de Views

As Views foram desenvolvidas com o objetivo de simplificar consultas SQL e facilitar a visualização das informações do sistema.

As Views implementadas utilizam operações JOIN para reunir dados de múltiplas tabelas em uma única consulta.

3.3.1 View Listar Fornecedor

A View **listar fornecedor** foi criada para exibir informações resumidas dos fornecedores cadastrados no sistema.

```
12  --VIEW LISTAR FORNECEDOR
13
14  create or replace view listar_Fornecedor as
15  select f.id_fornecedor, upper(f.nome_fornecedor) as nome_fornecedor, upper(f.
    email_fornecedor) as email, f.telefone_fornecedor, f.
    data_cadastro_fornecedor
16  |   from fornecedor f;
17
```

Figura 9 – View listar fornecedor

3.3.2 View Itens Comprados

A View **itens comprado estoque** realiza a junção, através de join entre as tabelas Produto, Categoria, Item Compra, Nota Compra e Fornecedor, permitindo visualizar os produtos comprados e seus respectivos fornecedores.

```

19  --VIEW ITENS COMPRADOS
20  create or REPLACE view itens_Comprado_Estoque as
21  select UPPER (f.nome_fornecedor) nome_fornecedor,nc.data_compra, ic.
    valor_unitario_item,ic.quantidade_produto_comprado, upper(p.nome_produto)
    nome_produto,nc.id_nota_compra,s.descricao,p.id_produto,upper(c.
    nome_categoria) nome_categoria
22  from produto p
23      inner join categoria c
24      on c.cod_categoria=p.id_categoria
25      inner join item_compra ic
26      on ic.id_produto=p.id_produto
27      inner join nota_compra nc
28      on ic.id_nota_compra=nc.id_nota_compra
29      inner join fornecedor f
30      on f.id_fornecedor = nc.id_fornecedor
31      inner join status s
32      on s.id_status = nc.id_status
33

```

Figura 10 – View itens comprado estoque

3.3.3 View Itens Vendidos

A View **itens vendidos** foi criada para visualizar informações relacionadas às vendas realizadas no sistema, incluindo cliente, produto, quantidade vendida e saldo de cashback.

```

34  --VIEW ITENS VENDIDOS
35  create or replace view itens_Vendidos as
36  select p.id_produto as ID_Produto,iv.preco_unitario_venda as valor_Item,
    UPPER(p.nome_produto) PRODUTO,iv.quantidade_produto_vendido as
    PRODUTOS_VENDIDOS, nv.data_venda as data,nv.valor_total_venda as Valor,
    upper(c.nome_cliente) CLIENTE,c.saldo_cashback, nv.id_nota_venda as ID_Nota,
    nv.valor_cashback_gerado,nv.valor_cashback_utilizado
37  from produto p
38      inner join item_venda iv
39      on iv.id_produto = p.id_produto
40      inner join nota_venda nv
41      on nv.id_nota_venda = iv.id_nota_venda
42      inner join cliente c
43      on c.id_cliente = nv.id_cliente
44

```

Figura 11 – View itens vendidos

3.3.4 View Produtos

A View **produtos** realiza a junção entre Produto e Categoria, permitindo listar os produtos juntamente com suas categorias, estoque atual e preço unitário.

```
48  --VIEW LISTAR PRODUTOS
49  ✓ CREATE OR REPLACE VIEW public.produtos
50      AS
51  ✓ SELECT upper(p.nome_produto::text) AS produto,
52          p.preco_unitario,
53          p.estoque_atual,
54          p.id_produto AS codigo,
55          upper(c.nome_categoria::text) AS categoria
56  ✓ FROM produto p
57      | JOIN categoria c ON c.cod_categoria = p.id_categoria;
58
```

Figura 12 – View Produtos

3.4 Uso de Procedures

As Procedures foram implementadas com o objetivo de automatizar operações importantes dentro do banco de dados.

3.4.1 Procedure Atualizar Cliente

A Procedure **atualizar cliente** foi desenvolvida para atualizar automaticamente os dados cadastrais do cliente, incluindo nome, e-mail e saldo de cashback.

```

226 --Procedure para atualizar os dados de um cliente
227 drop procedure atualizar_cliente(INT, VARCHAR, VARCHAR)
228 ✓ CREATE OR REPLACE PROCEDURE atualizar_cliente(
229     p_id_cliente INT,
230     p_nome VARCHAR,
231     p_email VARCHAR,
232     p_saldo_cashback decimal
233 )
234 LANGUAGE plpgsql
235 AS $$
236 ✓ BEGIN
237
238     UPDATE cliente
239     ✓ SET nome_cliente = p_nome,
240         email_cliente = p_email,
241         saldo_cashback = p_saldo_cashback
242     WHERE id_cliente = p_id_cliente;
243
244 END;
245 $$;
246

```

Figura 13 – Procedure Atualizar Cliente

3.4.2 Procedure Baixar Estoque

A Procedure **baixar estoque** é responsável por diminuir automaticamente a quantidade de produtos disponíveis após a realização de uma venda.

```

192  --PROCEDURE PARA DIMINUIR O ESTOQUE DE UM PRODUTO
193  CREATE OR REPLACE PROCEDURE baixar_estoque(
194      p_id_produto INT,
195      p_quantidade INT
196  )
197  LANGUAGE plpgsql
198  AS $$
199  BEGIN
200
201      UPDATE produto
202      SET estoque_atual = estoque_atual - p_quantidade
203      WHERE id_produto = p_id_produto;
204
205  END;
206  $$;
207

```

Figura 14 – Procedure Baixar Estoque

3.4.3 Procedure Aumentar Estoque

A Procedure **aumentar estoque** realiza o incremento automático do estoque após novas compras de produtos.

```

210  --PROCEDURE PARA AUMENTAR O ESTOQUE DE UM PRODUTO
211  CREATE OR REPLACE PROCEDURE aumentar_estoque(
212      p_id_produto INT,
213      p_quantidade INT
214  )
215  LANGUAGE plpgsql
216  AS $$
217  BEGIN
218
219      UPDATE produto
220      SET estoque_atual = estoque_atual + p_quantidade
221      WHERE id_produto = p_id_produto;
222
223  END;
224  $$;
225

```

Figura 15 – Procedure Aumentar Estoque

3.5 Uso de Functions

As Functions foram implementadas para realização de cálculos automáticos diretamente no banco de dados.

3.5.1 Function Calcular Faturamento Total

A Function **calcular faturamento total()** realiza a soma automática de todas as vendas registradas no sistema.

```
251  --FUNCTION PARA CALCULAR O FATURAMENTO TOTAL
252  CREATE OR REPLACE FUNCTION calcular_faturamento_total()
253  RETURNS NUMERIC
254  LANGUAGE plpgsql
255  AS $$
256  DECLARE
257  |   v_total NUMERIC;
258  BEGIN
259  |
260  |   SELECT COALESCE(SUM(valor_total_venda), 0)
261  |   INTO v_total
262  |   FROM nota_venda;
263  |
264  |   RETURN v_total;
265  |
266  END;
267  $$;
268
```

Figura 16 – Function Calcular Faturamento Total()

3.5.2 Function Calcular Valor Total do Estoque

A Function **calcular valor total estoque()** calcula automaticamente o valor financeiro total dos produtos armazenados no estoque.

```

270  --FUNCTION PARA CALCULAR O VALOR TOTAL DO ESTOQUE
271  CREATE OR REPLACE FUNCTION calcular_valor_total_estoque()
272  RETURNS NUMERIC
273  LANGUAGE plpgsql
274  AS $$
275  DECLARE
276  |   v_total NUMERIC;
277  BEGIN
278
279  |   SELECT COALESCE(SUM(preco_unitario * estoque_atual), 0)
280  |   INTO v_total
281  |   FROM produto;
282
283  |   RETURN v_total;
284
285  END;
286  $$;
287  SELECT calcular_valor_total_estoque();

```

Figura 17 – Function Calcular valor total Estoque()

4 Programação utilizando Java

O Sistema de Varejo de Produtos (SVP) foi desenvolvido com a utilização da linguagem de programação Java, utilizando o Eclipse como IDE (Integrated Development Environment) ou ambiente de desenvolvimento integrado para fazer conexão e comunicação entre a aplicação e o banco através da API JDBC (Java Database Connectivity) a qual permite a execução de operações SQL diretamente a partir do código Java. Deve-se ressaltar, que durante a implementação da aplicação foi aplicado os princípios de SOLID, garantindo o desacoplamento e permitindo maior manutenção ao código, juntamente com possíveis melhorias ao sistema.

4.1 Integração com Banco de Dados

4.1.1 Conexão da Classe em Java com o Banco de Dados

```
1 package repository;
2
3 import java.sql.Connection;
4
5
6
7 public class PostgreSQL implements ConexaoBanco{
8     private static final String URL= "jdbc:postgresql://localhost:5432/EmpresaVarejo";
9     private static final String USER= "postgres";
10    private static final String PASSWORD= "";
11
12    @Override
13    public Connection getConnection() throws SQLException{
14        return DriverManager.getConnection(URL,USER,PASSWORD);
15    }
16 }
17
```

Figura 18 – Classe em Java para estabelecer Conexão

A Figura 18 apresenta uma classe Java denominada PostgreSQL, responsável por implementar a interface ConexaoBanco. Nessa implementação, é definido o método getConnection(), que estabelece a conexão entre a aplicação Java e o banco de dados PostgreSQL.

Para isso, a classe utiliza o driver JDBC do PostgreSQL em conjunto com a classe DriverManager, responsável por gerenciar e fornecer a conexão com o banco. As bibliotecas necessárias são importadas para possibilitar a execução de operações SQL dentro da aplicação Java.

4.2 Operações CRUD + View

4.2.1 Inserção de Dados

```
13 public class ClienteRepositoryPostgres implements RepositoryBancoCliente {
14     private ConexaoBanco banco;
15
16     public ClienteRepositoryPostgres(ConexaoBanco banco) {
17         super();
18         this.banco = banco;
19     }
20
21     @Override
22     public void salvar(Cliente entidade) {
23         String sql = "INSERT INTO public.cliente (nome_cliente, cpf_cliente, email_cliente, data_cadastro) VALUES (?, ?, ?, ?)";
24         try (PreparedStatement preparar = banco.getConnection().prepareStatement(sql)) {
25             preparar.setString(1, entidade.getNome());
26             preparar.setString(2, entidade.getCpf());
27             preparar.setString(3, entidade.getEmail());
28             preparar.setDate(4, java.sql.Date.valueOf(LocalDate.now()));
29             preparar.executeUpdate();
30             preparar.close();
31         } catch (SQLException e) {
32             // TODO: handle exception
33         }
34     }
35 }
36
```

Figura 19 – Comando SQL INSERT em Java

O trecho do código selecionado na Figura 19 explica brevemente um comando de código SQL através de uma String em Java, onde esse trecho do código insere na tabela Cliente do banco os dados do cliente, passando valores que serão aceitos pelo terminal da IDE.

4.2.2 Exclusão de Dados

```
@Override
public void deletar(Integer id) {
    String sql = "DELETE FROM public.cliente WHERE id_cliente = ?";

    try (PreparedStatement preparar = banco.getConnection().prepareStatement(sql)) {
        preparar.setInt(1, id);
        preparar.executeUpdate();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
```

Figura 20 – Método em Java para exclusão de Cliente

A Figura 20 apresenta o método responsável pela exclusão de registros da entidade Cliente na aplicação Java. A operação é realizada por meio de um comando SQL DELETE, que remove do banco de dados o cliente correspondente ao identificador informado como parâmetro.

Para garantir a segurança da operação, é utilizada a classe PreparedStatement, permitindo a parametrização da consulta e evitando problemas relacionados à injeção de SQL. Após a definição do valor do parâmetro, o método executeUpdate() é executado para efetivar a exclusão do registro na tabela cliente.

Dessa forma, a aplicação consegue realizar a remoção de clientes de maneira segura, organizada e integrada ao banco de dados PostgreSQL por meio da API JDBC.

4.2.3 Consulta de Dados (com JOIN e View)

```
@Override
public List<ListarProdutos> listar() {
    List<ListarProdutos> lista = new ArrayList<>();
    String sql = "select * from produtos";
    try (PreparedStatement preparar = banco.getConnection().prepareStatement(sql);
        ResultSet rs = preparar.executeQuery()) {
        while (rs.next()) {
            ListarProdutos entidade = new ListarProdutos();
            entidade.setNome_categoria(rs.getString("categoria"));
            entidade.setPreco(rs.getBigDecimal("preco_unitario"));
            entidade.setNome_produto(rs.getString("produto"));
            entidade.setEstoque(rs.getInt("estoque_atual"));
            entidade.setId_produto(rs.getInt("codigo"));
            lista.add(entidade);
        }
        return lista;
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return new ArrayList<>();
}
```

Figura 21 – Método em Java para executar um View

```

48  --VIEW LISTAR PRODUTOS
49  v CREATE OR REPLACE VIEW public.produtos
50  AS
51  v SELECT upper(p.nome_produto::text) AS produto,
52      p.preco_unitario,
53      p.estoque_atual,
54      p.id_produto AS codigo,
55      upper(c.nome_categoria::text) AS categoria
56  v FROM produto p
57      JOIN categoria c ON c.cod_categoria = p.id_categoria;
58

```

Figura 22 – View executada pelo SGBD

A Figura 21 retrata um trecho de um método do código em Java para fazer consultas a tabela de clientes e contas, no método é executado um comando SQL a qual aciona uma View no SGBD, onde a Figura 22 demonstra o JOIN a qual faz a junção dos dados das duas tabelas: Produto e Categoria.

```

@Override
public List<RelatorioCompraDTO> listarCompra() {

    List<RelatorioCompraDTO> lista = new ArrayList<>();

    String sql = "SELECT * FROM itens_comprado_estoque";

    try {
        PreparedStatement ps = banco.getConnection().prepareStatement(sql);
        ResultSet rs = ps.executeQuery()
    } {

        while (rs.next()) {

            RelatorioCompraDTO dto = new RelatorioCompraDTO();

            dto.setNomeFornecedor(rs.getString("nome_fornecedor"));
            dto.setValorItem(rs.getBigDecimal("valor_unitario_item"));
            dto.setQuantidadeProdutoComprado(rs.getInt("quantidade_produto_comprado"));
            dto.setNomeProduto(rs.getString("nome_produto"));
            dto.setEstoqueAtual(rs.getInt("estoque_atual"));
            dto.setNomeCategoria(rs.getString("nome_categoria"));

            lista.add(dto);

        }

    } catch (SQLException e) {
        e.printStackTrace();
    }

    return lista;
}

```

Figura 23 – Método em Java para executar um View

```

19 --VIEW ITENS COMPRADOS
20 create or REPLACE view itens_Comprado_Estoque as
21 select UPPER (f.nome_fornecedor) nome_fornecedor,nc.data_compra, ic.
  valor_unitario_item,ic.quantidade_produto_comprado, upper(p.nome_produto)
  nome_produto,nc.id_nota_compra,s.descricao,p.id_produto,upper(c.
  nome_categoria) nome_categoria
22 from produto p
23     inner join categoria c
24         on c.cod_categoria=p.id_categoria
25     inner join item_compra ic
26         on ic.id_produto=p.id_produto
27     inner join nota_compra nc
28         on ic.id_nota_compra=nc.id_nota_compra
29     inner join fornecedor f
30         on f.id_fornecedor = nc.id_fornecedor
31     inner join status s
32         on s.id_status = nc.id_status
33

```

Figura 24 – View executada pelo SGBD

A Figura 23 apresenta o método responsável por consultar a View itens comprado estoque por meio da aplicação Java. Para isso, é utilizada uma instrução SQL do tipo SELECT, executada através das classes PreparedStatement e ResultSet da API JDBC. Os dados retornados pela consulta são percorridos e armazenados em objetos da classe RelatorioCompraDTO, que posteriormente são adicionados a uma lista para serem utilizados pela aplicação.

Já a Figura 24 apresenta a definição da View itens comprado estoque no PostgreSQL. Essa View foi criada utilizando operações de INNER JOIN entre as tabelas produto, categoria, item compra, nota compra e fornecedor, consolidando informações relacionadas às compras realizadas e ao estoque atual dos produtos. Dessa forma, a aplicação consegue obter os dados necessários para a geração de relatórios por meio de uma única consulta, simplificando o acesso às informações e melhorando a organização do código.

4.2.4 Alteração de Dados

```

@Override
public void atualizar(Categoria entidade) {
    String sql = ""
        UPDATE public.categoria
        SET nome_categoria=?, descricao=?
        WHERE ?
        ""

    try (PreparedStatement preparar = banco.getConnection().prepareStatement(sql)) {
        preparar.setString(1, entidade.getNome());
        preparar.setString(2, entidade.getDescricao());
        preparar.setInt(3, entidade.getId());
        preparar.executeUpdate();
        preparar.close();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

```

Figura 25 – Atualizar dados da Categoria em Java

A Figura 25 apresenta o método responsável pela atualização dos dados da entidade Categoria na aplicação Java. A operação é realizada por meio de um comando SQL

UPDATE, que modifica os campos nome categoria e descricao de um registro específico na tabela categoria.

Para garantir maior segurança e evitar problemas como injeção de SQL, a consulta utiliza a classe PreparedStatement, que permite a parametrização dos valores a serem atualizados. Após a definição dos parâmetros, o método executeUpdate() é executado para efetivar a alteração no banco de dados. Dessa forma, a aplicação mantém os dados cadastrados atualizados de maneira segura e eficiente.

5 Procedimentos Avançados

5.1 Uso de Function

```
@Override
public BigDecimal listarFaturamento() {
    String sql = "SELECT calcular_faturamento_total()";

    try (PreparedStatement ps = banco.getConnection().prepareStatement(sql);
        ResultSet rs = ps.executeQuery()) {

        if (rs.next()) {
            return rs.getBigDecimal(1);
        }

    } catch (SQLException e) {
        e.printStackTrace();
    }

    return BigDecimal.ZERO;
}
```

Figura 26 – Código em Java para chamada do SGBD

```
251 --FUNCTION PARA CALCULAR O FATURAMENTO TOTAL
252 CREATE OR REPLACE FUNCTION calcular_faturamento_total()
253 RETURNS NUMERIC
254 LANGUAGE plpgsql
255 AS $$
256 DECLARE
257     v_total NUMERIC;
258 BEGIN
259
260     SELECT COALESCE(SUM(valor_total_venda), 0)
261     INTO v_total
262     FROM nota_venda;
263
264     RETURN v_total;
265
266 END;
267 $$;
268
```

Figura 27 – Comandos de uma Function no SGBD

As Figuras 26 e 27 demonstram a integração entre a aplicação Java e uma Function criada no PostgreSQL para cálculo do faturamento total da empresa. A estratégia foi adotada

para centralizar a regra de negócio no banco de dados, permitindo que o cálculo seja executado diretamente pelo SGBD e retornado à aplicação de forma simplificada.

Na Figura 26, é apresentado o método Java responsável por realizar a chamada da função calcular faturamento total(). A consulta é executada utilizando JDBC por meio das classes PreparedStatement e ResultSet, recuperando o valor retornado pela função e armazenando-o em uma variável do tipo BigDecimal, adequada para operações monetárias.

Já a Figura 27 apresenta a implementação da função no PostgreSQL. A função realiza uma consulta na tabela nota venda, somando todos os valores registrados no campo valor total venda através da função de agregação SUM(). O comando COALESCE() é utilizado para garantir que, caso não existam registros na tabela, o resultado retornado seja zero. Dessa forma, a aplicação consegue obter o faturamento total de maneira eficiente, reutilizável e com baixo acoplamento entre a camada de aplicação e a camada de persistência de dados.

5.2 Uso do Procedure

```
@Override
public void atualizar(Cliente entidade) {
    String sql = ""
        CALL atualizar_cliente(?, ?, ?, ?)
        "";

    try (CallableStatement preparar = banco.getConnection().prepareCall(sql)) {

        preparar.setString(2, entidade.getNome());
        preparar.setString(3, entidade.getEmail());
        preparar.setBigDecimal(4, entidade.getCashback());
        preparar.setInt(1, entidade.getId());
        preparar.executeUpdate();
        preparar.close();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
```

Figura 28 – Comando de Procedure executado no JDBC


```

226 --Procedure para atualizar os dados de um cliente
227 drop procedure atualizar_cliente(INT, VARCHAR, VARCHAR)
228 CREATE OR REPLACE PROCEDURE atualizar_cliente(
229     p_id_cliente INT,
230     p_nome VARCHAR,
231     p_email VARCHAR,
232     p_saldo_cashback decimal
233 )
234 LANGUAGE plpgsql
235 AS $$
236 BEGIN
237
238     UPDATE cliente
239     SET nome_cliente = p_nome,
240         email_cliente = p_email,
241         saldo_cashback = p_saldo_cashback
242     WHERE id_cliente = p_id_cliente;
243
244 END;
245 $$;
246

```

Figura 29 – Comandos de uma Function no SGBD

As Figuras 28 e 29 apresentam a utilização de uma Procedure no PostgreSQL para realizar a atualização dos dados de clientes cadastrados no sistema. Essa abordagem permite centralizar a lógica de alteração diretamente no banco de dados, promovendo maior organização, reutilização de código e facilidade de manutenção.

Na Figura 28, é demonstrado o método Java responsável por executar a Procedure atualizar cliente. A chamada é realizada por meio da classe CallableStatement, específica para a execução de procedimentos armazenados no banco de dados. Os atributos do objeto Cliente são enviados como parâmetros para a Procedure, que será responsável por efetuar a atualização do registro correspondente.

A Figura 29 apresenta a implementação da Procedure no PostgreSQL. O procedimento recebe como parâmetros o identificador do cliente, seu nome, e-mail e saldo de cashback, executando um comando UPDATE na tabela cliente. Dessa forma, a atualização dos dados é realizada diretamente pelo SGBD, garantindo maior padronização das operações e reduzindo a complexidade da camada de aplicação.

6 Conclusão

O desenvolvimento do Sistema de Gerenciamento de Varejo possibilitou a consolidação prática de diversos conceitos fundamentais relacionados a banco de dados, programação orientada a objetos e desenvolvimento de aplicações Java. Utilizando a linguagem Java em conjunto com a API JDBC, foi possível implementar um sistema funcional capaz de realizar operações de cadastro, consulta, atualização e exclusão de dados, além do gerenciamento de compras, vendas, clientes, fornecedores, produtos e categorias.

A modelagem conceitual e lógica foi cuidadosamente elaborada para representar os processos de um ambiente varejista, garantindo a integridade e a consistência dos dados armazenados no PostgreSQL. A utilização de relacionamentos entre as entidades permitiu o controle adequado do estoque, do histórico de compras e vendas, bem como a geração de relatórios gerenciais para apoio à tomada de decisão.

Durante o desenvolvimento, foram aplicados recursos avançados do SGBD PostgreSQL, como Views, Functions e Procedures, possibilitando a centralização de regras de negócio e a otimização de consultas. A integração dessas funcionalidades com a aplicação Java, por meio do JDBC, permitiu uma comunicação eficiente entre a camada de persistência e a camada de aplicação.

Além disso, foram utilizadas boas práticas de programação, como a separação das responsabilidades em camadas, a utilização de interfaces e padrões de acesso a dados, bem como o emprego de PreparedStatement e CallableStatement para maior segurança e desempenho na execução das operações SQL.

Embora o sistema atenda aos requisitos propostos, existem possibilidades de evolução, como a implementação de uma interface gráfica, controle de usuários e permissões, geração de relatórios em PDF, emissão de notas fiscais, autenticação de usuários e integração com sistemas externos. Também podem ser incorporados mecanismos mais avançados de tratamento de exceções, auditoria de operações e monitoramento do estoque em tempo real.

Conclui-se que o projeto atingiu seus objetivos acadêmicos e práticos, permitindo a aplicação integrada dos conhecimentos adquiridos ao longo da disciplina. O sistema desenvolvido representa uma solução funcional para o gerenciamento de operações de varejo, demonstrando a importância do planejamento, da modelagem de dados e da integração entre banco de dados e desenvolvimento de software para a construção de aplicações robustas e escaláveis.